pyFAI Peak-Finding and Scattering Reduction Algorithms

Source docs used:
- pyFAI stable docs: https://pyfai.readthedocs.io/en/stable/
- Calibration peak-picking: https://pyfai.readthedocs.io/en/stable/calibration.html
- peakfinder CLI: https://pyfai.readthedocs.io/en/stable/man/peakfinder.html
- Signal separation: https://pyfai.readthedocs.io/en/stable/usage/tutorial/Separation/index.html
- Laue diffraction peak identification:
https://pyfai.readthedocs.io/en/stable/usage/tutorial/Separation/Laue.html
- PeakFinder8 GPU tutorial:
https://pyfai.readthedocs.io/en/stable/usage/tutorial/Separation/Peakfinder8.html
- pyFAI API: https://pyfai.readthedocs.io/en/stable/api/pyFAI.html
- Grazing incidence / fiber diffraction:
https://pyfai.readthedocs.io/en/stable/usage/tutorial/FiberGrazingIncidence.html

Scope: pyFAI is mainly a scattering-image reduction library. Its peak-relevant tools fall into four groups: calibration peak picking on Debye-Scherrer rings, GPU Bragg peak detection, sigma-clipping background separation, and geometry-aware integration that prepares SAXS/WAXS/GISAXS/GIWAXS data for downstream peak detection.

1. AzimuthalIntegrator.integrate1d: reduce 2D detector images to 1D peak curves

Functionality: AzimuthalIntegrator.integrate1d converts an area-detector image into a 1D scattering curve such as intensity versus q or two-theta. This is not a peak finder by itself, but it is the standard pyFAI step before using 1D peak algorithms such as scipy.signal.find_peaks.

Use case: SAXS/WAXS/XRPD data with rings or arcs. Use a calibrated .poni geometry file, mask invalid detector pixels, integrate to I(q), then find peak centers, widths, and prominences on the 1D curve.

Code snippet:
```python
import pyFAI
from scipy.signal import find_peaks, peak_widths


def integrate_then_find_1d_peaks(image, poni_file, mask=None, npt=2000):
    ai = pyFAI.load(poni_file)
    result = ai.integrate1d(
        image,
        npt=npt,
        unit="q_A^-1",
        mask=mask,
        error_model="poisson",
        method=("bbox", "csr", "cython"),
    )

    q = result.radial
    intensity = result.intensity
    peaks, props = find_peaks(
        intensity,
        prominence=0.03 * intensity.max(),
        distance=8,
    )
    widths = peak_widths(intensity, peaks, rel_height=0.5)

    return {
        "q": q,
        "intensity": intensity,
        "peaks": [
            {
                "q": float(q[p]),
                "intensity": float(intensity[p]),
                "prominence": float(props["prominences"][i]),
                "width_samples": float(widths[0][i]),
            }
            for i, p in enumerate(peaks)
```

```
        ],
    }
```

2. AzimuthalIntegrator.sigma_clip_ng: background and amorphous-signal separation

Functionality: sigma_clip_ng iteratively integrates with variance propagation and rejects pixels whose intensities differ from the radial-bin mean by more than a threshold times the standard deviation. The docs describe this as useful for extracting background or amorphous isotropic scattering out of Bragg peaks.

Use case: estimate smooth background in images with spotty crystalline peaks, remove positive outliers, or compare normal integration to clipped integration to isolate peaks. Good preprocessing for peak finding because Bragg spots become outliers above the sigma-clipped background.

Code snippet:
```python
import pyFAI
import numpy as np


def estimate_background_with_sigma_clipping(image, poni_file, mask=None, npt=2000):
    ai = pyFAI.load(poni_file)
    clipped = ai.sigma_clip_ng(
        image,
        npt=npt,
        unit="q_A^-1",
        mask=mask,
        thres=5.0,
        max_iter=5,
        error_model="poisson",
        method=("no", "csr", "cython"),
    )

    normal = ai.integrate1d(image, npt=npt, unit="q_A^-1", mask=mask)
    excess = normal.intensity - clipped.intensity
    return clipped.radial, clipped.intensity, excess
```

3. pyFAI.opencl.peak_finder.OCL_PeakFinder: GPU Bragg peak detection

Functionality: OCL_PeakFinder detects Bragg peaks on detector images using OpenCL. The Laue tutorial builds a sparse integrator, creates OCL_PeakFinder with the integrator lookup table, and calls peakfinder. Results include peak index, integrated intensity, sigma, and centroid coordinates pos0/pos1.

Use case: single-crystal diffraction, Laue diffraction, serial crystallography, or spotty scattering frames where 2D peaks must be detected directly without first collapsing to 1D. Requires an OpenCL device; GPU gives best performance.

Code snippet:
```python
import pyFAI
from pyFAI.opencl.peak_finder import OCL_PeakFinder


def find_bragg_peaks_opencl(image, poni_file, mask=None, npt=2000):
    ai = pyFAI.load(poni_file)
    unit = "q_nm^-1"
    error_model = "poisson"

    engine = ai.setup_sparse_integrator(
        image.shape,
        npt,
        mask=mask,
        unit=unit,
        split="no",
```

```
    )
    radius2d = ai.array_from_unit(image.shape, unit=unit)
    peak_finder = OCL_PeakFinder(
        engine.lut,
        image.size,
        unit=unit,
        bin_centers=engine.bin_centers,
        radius=radius2d,
        mask=mask,
    )

    peaks = peak_finder.peakfinder(
        image,
        error_model=error_model,
        patch_size=15,
        connected=15,
    )
    return peaks
```

Output interpretation: use peaks["pos0"] and peaks["pos1"] as sub-pixel centroid coordinates; peaks["intensity"] as integrated peak signal; peaks["sigma"] as uncertainty-like signal descriptor.

4. PeakFinder8 workflow: threshold, connected pixels, patch integration, centroid

Functionality: pyFAI's PeakFinder8 implementation follows a frame-level peak workflow: integrate with uncertainty propagation, sigma-clip background, threshold high pixels, require local maximum and connected high pixels in a 3x3 or 5x5 patch, subtract background, integrate signal over the patch, and return peak index, signal, and center of mass.

Use case: real-time assessment of serial crystallography frames or any spotty diffraction image where the number and quality of Bragg peaks matter. The algorithm is GPU-oriented in pyFAI.

Code snippet:
```python
from pyFAI.opencl.peak_finder import OCL_PeakFinder


def run_peakfinder8(peak_finder, image):
    peaks = peak_finder.peakfinder8(
        image,
        error_model="azimuthal",
        cycle=3,
        cutoff_peak=3,
        noise=2,
        connected=9,
        patch_size=5,
    )
    return [
        {
            "pixel_index": int(row["index"]),
            "intensity": float(row["intensity"]),
            "sigma": float(row["sigma"]),
            "row": float(row["pos0"]),
            "col": float(row["pos1"]),
        }
        for row in peaks
    ]
```

Tuning notes: increase patch_size for broad spots; increase connected to require larger connected bright regions; increase cutoff_peak to reduce false positives; tune noise when weak detector noise creates spurious high pixels.

5. peakfinder CLI / sparsify-Bragg: command-line Bragg peak detection

Functionality: the peakfinder application counts Bragg peaks in images. Docs define Bragg peaks

as local maxima of background-subtracted signal; peaks are integrated, variance is propagated, and centroids are reported. Background comes from iterative sigma-clipping in polar space.

Use case: batch peak extraction into HDF5 from image files when Python scripting is not desired. Requires OpenCL.

CLI snippet:
```bash
sparsify-Bragg   --poni geometry.poni   --mask detector_mask.edf   --output peaks.h5   --bins 2000   --unit q_nm^-1   --cycle 5   --cutoff-clip 5   --error-model poisson   --cutoff-pick 3 --noise 2   --patch-size 5   --connected 9   frame_0001.edf frame_0002.edf
```

Parameter meanings: cutoff-clip controls sigma-clipping outlier threshold; cutoff-pick controls high-pixel SNR threshold; patch-size sets neighborhood used for peak integration; connected sets how many high pixels must be connected to call a peak.

6. Calibration peak-picking: massif detection

Functionality: Massif detection subtracts a blurred image from the original image and finds contiguous positive regions. Those regions correspond to groups of peaks on calibration rings. The API Massif.find_peaks starts from a seed, finds a maximum, calculates region extension, and extracts neighboring peaks.

Use case: calibration with Debye-Scherrer rings from calibrants such as LaB6, CeO2, silicon, or silver behenate. Best when a selected ring area contains many local maxima belonging to the same ring group.

Code snippet:
```python
from pyFAI.massif import Massif


def pick_calibration_peaks_massif(image, seed_yx, mask=None, nmax=200):
    massif = Massif(image, mask=mask, median_prefilter=True)
    nearest = massif.nearest_peak(seed_yx)
    peaks = massif.find_peaks(nearest, nmax=nmax)
    return peaks
```

Use seed_yx as user-picked approximate location on a calibrant ring. The output peak list can become control points for geometry refinement.

7. Calibration peak-picking: blob detection / Difference of Gaussians

Functionality: BlobDetection uses Difference of Gaussian and a Gaussian pyramid. It detects keypoints across image location and scale, then refines positions using Hessian-based second-order refinement or Savitzky-Golay checks.

Use case: calibration images where peaks vary in width or where scale matters. Useful when ring peaks appear as blob-like spots and the approximate spot size is not constant.

Code snippet:
```python
from pyFAI.blob_detection import BlobDetection


def pick_calibration_peaks_blob(image, area_mask, mask=None):
    detector = BlobDetection(
        image,
        cur_sigma=0.25,
        init_sigma=0.5,
        dest_sigma=1.0,
        scale_per_octave=2,
        mask=mask,
    )
    detector.process()
    peaks = detector.peaks_from_area(
```

```
        area_mask,
        refine=True,
        Imin=None,
        dmin=3.0,
    )
    return peaks
```

Tuning notes: cur_sigma represents estimated smoothing in input data; init_sigma sets first search scale; scale_per_octave controls scale sampling; dmin helps suppress duplicate nearby peaks.

8. Calibration peak-picking: steepest ascent and Monte Carlo sampling

Functionality: Steepest ascent searches from a seed toward the nearest local maximum, then applies sub-pixel optimization using a local gradient and Hessian. Monte Carlo sampling repeats steepest-ascent extraction from randomly selected seeds, optionally biased toward a known geometry/ring.

Use case: interactive calibration. Steepest ascent is useful when user supplies approximate ring points. Monte Carlo is useful when you need many peaks on a ring and can generate random seeds near the expected ring locus.

Pseudo-code snippet:
```python
import random


def monte_carlo_peak_seeds(expected_ring_pixels, n_samples=200):
    seeds = []
    for _ in range(n_samples):
        y, x = random.choice(expected_ring_pixels)
        seeds.append((y + random.uniform(-3, 3), x + random.uniform(-3, 3)))
    return seeds


def refine_seeds_with_massif(image, seeds, mask=None):
    from pyFAI.massif import Massif
    massif = Massif(image, mask=mask, median_prefilter=True)
    refined = []
    for seed in seeds:
        try:
            refined.append(massif.nearest_peak(seed))
        except Exception:
            pass
    return refined
```

9. Geometry refinement from picked peaks

Functionality: After calibration peak picking, each group is assigned to a Debye-Scherrer ring number and d-spacing. pyFAI refines six geometry parameters by least-squares fitting the difference between observed and expected 2theta positions. The docs identify scipy.optimize.slsqp as default optimization procedure.

Use case: turn picked peak coordinates into a .poni geometry file, then reuse that geometry for SAXS/WAXS/GISAXS/GIWAXS reduction and peak analysis.

Conceptual snippet:
```python
# Typical workflow, usually driven by pyFAI-calib / pyFAI-calib2:
# 1. Load calibrant image.
# 2. Pick peak groups on known calibrant rings.
# 3. Assign each peak group to ring index and d-spacing.
# 4. Refine detector distance, PONI position, rotations, and wavelength/energy if needed.
# 5. Save geometry to geometry.poni.
# 6. Use geometry.poni for integrate1d, integrate2d, sigma_clip_ng, or OCL_PeakFinder.
```

10. FiberIntegrator / grazing-incidence integration for GISAXS/GIWAXS peak profiles

Functionality: FiberIntegrator remaps grazing-incidence or fiber diffraction patterns into q_IP versus q_OOP maps and supports 1D integrations over selected q regions. This is not direct peak picking, but it creates physically meaningful line profiles where peak finding can be applied.

Use case: GISAXS/GIWAXS patterns where peaks/ridges should be analyzed along in-plane or out-of-plane reciprocal-space directions. Use integrate2d_grazing_incidence for maps, then integrate1d_grazing_incidence over q windows and apply 1D peak finding.

Code snippet:

```python
import numpy as np
from scipy.signal import find_peaks
from pyFAI.integrator.fiber import FiberIntegrator


def giwaxs_line_profile_peaks(image, poni_file, incident_angle_deg=0.12):
    fi = FiberIntegrator(poni_file)
    line = fi.integrate1d_grazing_incidence(
        data=image,
        sample_orientation=6,
        incident_angle=np.deg2rad(incident_angle_deg),
        npt_ip=1000,
        npt_oop=100,
        ip_range=(-7, 7),
        oop_range=(5.5, 6.25),
        vertical_integration=False,
        method=("no", "csr", "cython"),
    )

    x = line.integrated
    y = line.intensity
    peaks, props = find_peaks(y, prominence=0.03 * y.max(), distance=10)
    return [{"q_ip": float(x[p]), "intensity": float(y[p])} for p in peaks]
```

Caution from docs: grazing-incidence integration should normally avoid pixel splitting; if using bbox pixel splitting, use missing-wedge masking where appropriate.

11. Practical selection guide

Algorithm selection:
- 1D SAXS/WAXS curve peak finding: pyFAI integrate1d -> SciPy find_peaks.
- Spotty 2D Bragg images: OCL_PeakFinder.peakfinder or peakfinder8.
- Calibration rings: pyFAI-calib peak picking with Massif, BlobDetection, Steepest Ascent, or Monte Carlo sampling.
- Background/amorphous signal separation: sigma_clip_ng.
- GISAXS/GIWAXS directional peak profiles: FiberIntegrator integrate2d_grazing_incidence or integrate1d_grazing_incidence -> 1D peak finding.

KG node candidates:
AzimuthalIntegrator.integrate1d, AzimuthalIntegrator.sigma_clip_ng, OCL_PeakFinder, peakfinder8, Massif, BlobDetection, Steepest Ascent, Monte Carlo sampling, GeometryRefinement, FiberIntegrator, integrate2d_grazing_incidence, integrate1d_grazing_incidence, sparsify-Bragg, Debye-Scherrer rings, calibrant, .poni geometry file.